EXPERIMENT - 6

AIM:

To write a program to implement error detection and correction using HAMMING code concept, and make a test run to input data stream and verify error correction feature.

CODE:

```
def redundant_bits(m):
    r=0
    while 2**r < m+r+1:
        r+=1
    return r

def encode(data):
    m=len(data)
    r= rendundant_bits(m)
    print("Number of data bits:", m)
    print("Number of redundant bits: ", r)

    n= m+r
    h = ['0'] *(n+1)

    j=0
    for i in range(r):
        p=2**i
        parity=0
        for j in range(1,n+1):
            if j & p:
                parity ^= int(h[j])
        h[p]= str(parity)

    print("Redundant bit positions: " [2** i for iin
                                       range(r)])
    print("Hamming Code: ", ".join(h[1:]))

    return ".join h[1:])
```

```
def check(h):
    n= len(h)
    r=0

    while 2**r <=n :
        r+ = 1

    error = 0
    print ("\n Parity Checking: ")

    for i in range (r):
        p=2**i
        parity=0

        for j in range (1,n+1):
            if j & p :
                parity ^= int[h[j-1]]

        print ("Position", p, ":", parity)

        if parity! =0 :
            error+=p

    if error = =0 :
        print("\n No error detected ")
    else:
        print("Error has occured ")
        print('Error position: ",error)

        if h[error-1] = = '0':
            h[error-1]="1"
        else:
            h[error-1] = '0'
        print ("Error Corrected ")
        print ("Corrected Hamming Code:" ,"join (h))

    return h
```

```
def remove_parity(h):
    data = ''
    for i in range(1, len(h)+1):
        if i % (i-1):
            data += h[i-1]
    return data

def binary_to_text(data):
    text = ''
    for i in range(0, len(data), 8):
        byte = data[i:i+8]
        text += chr(int(byte,2))
    return text

text = input("Enter data to transmit:")
binary = ''.join(format(ord(c), '08b') for c in text)

print("\n Data:" text)
print("Binary:", binary)

channel = encode(binary)

with open("channel.txt", "w") as f:
    f.write(channel)

print("\n Data saved in Chanel .txt")

with open("chanel.txt", "r") as f:
    received = f.read()

print("\n Received data:")
print(received)

Change = inpt("\n Do you want to change the
    transmitted Data? (yes/no):")

if Change.lower() == "yes":
    pos = int(input("Enter position to Change:"))
    if pos >= 1 and pos <= len(received):
        h = list(received)
```

```
        if h[pos-1]=='0':
            h[pos-1]='1'
        else:
            h[pos-1]='0'

        received=''.join(h)
        print("Changed Data:")
        print(received)
    else:
        print("Invalid Position")
        exit()

hamming=list(received)
hamming=check(hamming)
data=remove_parity(hamming)
print("\n Data after removing redundant bits:")
print(data)
output=binary_to_text(data)
print("\n Output:",output)
```

INPUT: Enter data to transmit: HI
Do you want to change the transmitted data? (yes/no): no

OUTPUT: Data: HI
Binary: 0100100001001001
No. of data bits: 16
No. of redundant bits: 5
Redundant bit positions: [1,2,4,8,16]
Hamming code: 10011001100000100100 1
Data saved in channel.txt
Received Data:
10011001100000100100 1
Data after removing redundant bits:
0100100001001001

RESULT:

Thus the program is implemented using python for error correction & error detection using hamming code was executed successfully.